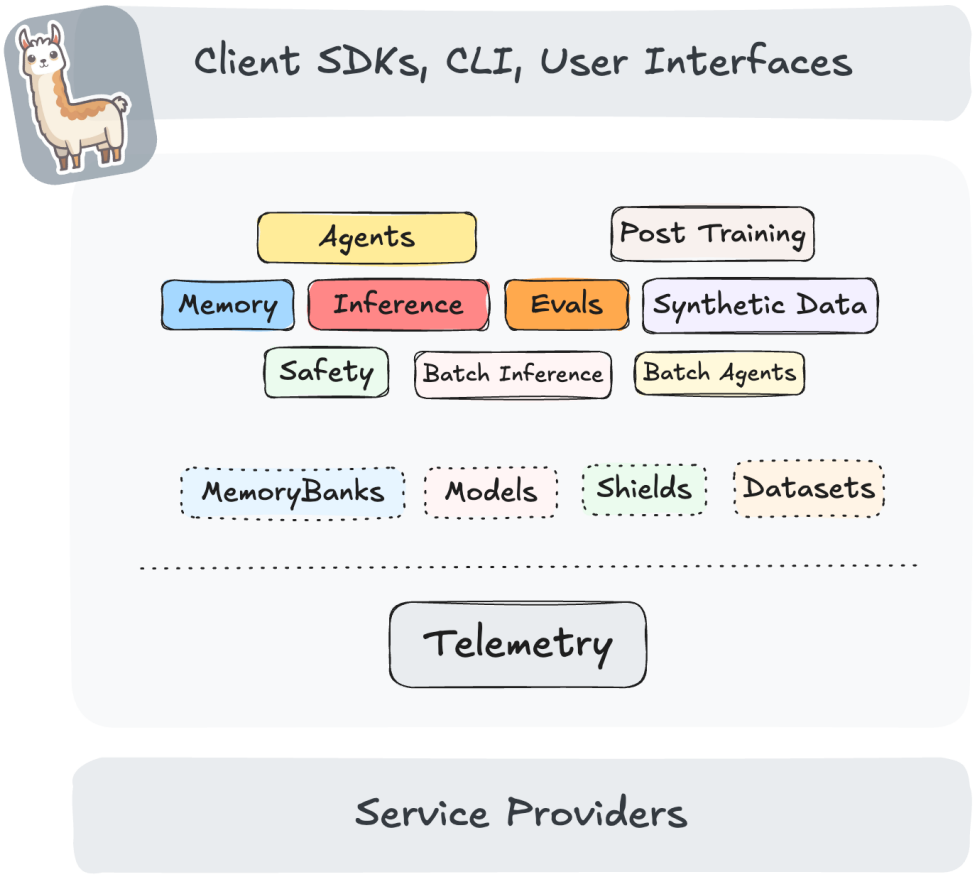
Open in Colab

Getting Started with Llama 4 in Llama Stack



Llama Stack defines and standardizes the set of core building blocks needed to bring generative AI applications to market. These building blocks are presented in the form of interoperable APIs with a broad set of Service Providers providing their implementations.

Read more about the project here: https://llama-stack.readthedocs.io/en/latest/index.html

In this guide, we will showcase how you can get started with using Llama 4 in Llama Stack.

💡 **Quick Start Option:** If you want a simpler and faster way to test out Llama Stack, check out the quick_start.ipynb notebook instead. It provides a streamlined experience for getting up and running in just a few steps.

1. Getting started with Llama Stack

1.1. Download Llama 4 Model

In this showcase, we will use run Llama 4 locally. Note you need 8xH100 GPU-host to run these models.

```
!pip install uv

MODEL="Llama-4-Scout-17B-16E-Instruct"
# get meta url from llama.com
!uv run --with llama-stack llama model download --source meta --model-id $MODEL -

model_id = f"meta-llama/{MODEL}"
```

1.2. Setup and Running a Llama Stack server

Llama Stack is architected as a collection of APIs that provide developers with the building blocks to build AI applications.

Llama stack is typically available as a server with an endpoint that you can make calls to. Partners like Together and Fireworks offer their own Llama Stack compatible endpoints.

In this showcase, we will start a Llama Stack server that is running locally.

```
import os
import subprocess
import time

!uv pip install requests

if "UV_SYSTEM_PYTHON" in os.environ:
  del os.environ["UV_SYSTEM_PYTHON"]

# this command installs all the dependencies needed for the llama stack server
!uv run --with llama-stack llama stack build --template meta-reference-gpu --imag

def run_llama_stack_server_background():
    log_file = open("llama_stack_server.log", "w")
    process = subprocess.Popen(
        f"uv run --with llama-stack llama stack run meta-reference-gpu --image-ty
        shell=True,
        stdout=log_file,
        stderr=log_file,
        text=True
    )

    print(f"Starting Llama Stack server with PID: {process.pid}")
    return process
```

```
def wait_for_server_to_start():
    import requests
    from requests.exceptions import ConnectionError
    import time

    url = "http://0.0.0.0:8321/v1/health"
    max_retries = 30
    retry_interval = 1

    print("Waiting for server to start", end="")
    for _ in range(max_retries):
        try:
            response = requests.get(url)
            if response.status_code == 200:
                print("\nServer is ready!")
                return True
        except ConnectionError:
            print(".", end="", flush=True)
            time.sleep(retry_interval)

    print("\nServer failed to start after", max_retries * retry_interval, "second
    return False


# use this helper if needed to kill the server
def kill_llama_stack_server():
    # Kill any existing llama stack server processes
    os.system("ps aux | grep -v grep | grep llama_stack.distribution.server.serve
```

Afficher la sortie masquée

1.3 Starting the Llama Stack Server

```
server_process = run_llama_stack_server_background()
assert wait_for_server_to_start()
```

1.4 Install and Configure the Client

Now that we have our Llama Stack server running locally, we need to install the client package to interact with it. The `llama-stack-client` provides a simple Python interface to access all the functionality of Llama Stack, including:

- Chat Completions (text and multimodal)
- Safety Shields
- Agent capabilities with tools like web search, RAG with Telemetry

- Evaluation and scoring frameworks

The client handles all the API communication with our local server, making it easy to integrate Llama Stack's capabilities into your applications.

In the next cells, we'll:

1. Install the client package
2. Initialize the client to connect to our local server

```
!pip install -U llama-stack-client
```

```
Using Python 3.10.16 environment at: /opt/homebrew/Caskroom/miniconda/base/en
Resolved 31 packages in 284ms
Audited 31 packages in 0.04ms
```

```
from llama_stack_client import LlamaStackClient

client = LlamaStackClient(
    base_url="http://0.0.0.0:8321",
)
```

Afficher la sortie masquée

Now that we have completed the setup and configuration, let's start exploring the capabilities of Llama 4!

2. Running Llama 4

2.1 Check available models

All the models available are programmatically accessible via the client.

```
from rich.pretty import pprint

print("Available models:")
for m in client.models.list():
    print(f"- {m.identifier}")
```

2.2 Run a simple chat completion with one of the models

We will test the client by doing a simple chat completion.

```
response = client.inference.chat_completion(
    model_id=model_id,
    messages=[
        {"role": "system", "content": "You are a friendly assistant."},
        {"role": "user", "content": "Write a two-sentence poem about llama."},
    ],
)

print(response.completion_message.content)
```

```
Here is a two-sentence poem about a llama:

With soft fur and gentle eyes, the llama roams with gentle surprise, a peacefu
```

2.3 Running multimodal inference

```
!curl -O https://raw.githubusercontent.com/meta-llama/llama-models/refs/heads/main/
```

```
from IPython.display import Image
Image("Llama_Repo.jpeg", width=256, height=256)
```

```
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Curren
                                 Dload  Upload   Total   Spent    Left  Speed
100  275k  100  275k    0     0   847k      0 --:--:-- --:--:-- --:--:--  845k
```

```
import base64
def encode_image(image_path):
    with open(image_path, "rb") as image_file:
        base64_string = base64.b64encode(image_file.read()).decode("utf-8")
        base64_url = f"{base64_string}"
        return base64_url
```

```
response = client.inference.chat_completion(
    messages=[
        {
            "role": "user",
            "content": [
                {
                    "type": "image",
                    "image": {
                        "url": {
                            "uri": encode_image("Llama_Repo.jpeg")
                        }
                    }
                },
                {
                    "type": "text",
                    "text": "How many different colors are those llamas? What are
                }
            ]
        }
    ],
    model_id=model_id,
    stream=False,
)

print(response.completion_message.content)
```

```
The image features three llamas, each with a distinct color. The llama on the

To determine the number of different colors present, we can count the unique l

1. White (two llamas)
2. Purple (one llama)
3. Blue (party hat)

Therefore, there are 3 different colors visible in the image: white, purple, a
```

2.4 Have a conversation

Maintaining a conversation history allows the model to retain context from previous interactions. Use a list to accumulate messages, enabling continuity throughout the chat session.

```
from termcolor import cprint

questions = [
    "Who was the most famous PM of England during world war 2 ?",
    "What was his most famous quote ?"
```

```
]


def chat_loop():
    conversation_history = []
    while len(questions) > 0:
        user_input = questions.pop(0)
        if user_input.lower() in ["exit", "quit", "bye"]:
            cprint("Ending conversation. Goodbye!", "yellow")
            break

        user_message = {"role": "user", "content": user_input}
        conversation_history.append(user_message)

        response = client.inference.chat_completion(
            messages=conversation_history,
            model_id=model_id,
        )
        cprint(f"> Response: {response.completion_message.content}", "cyan")

        assistant_message = {
            "role": "assistant",  # was user
            "content": response.completion_message.content,
            "stop_reason": response.completion_message.stop_reason,
        }
        conversation_history.append(assistant_message)


chat_loop()
```

```
> Response: The most famous Prime Minister of England during World War 2 was \

Churchill played a crucial role in rallying the British people during the war

Churchill's leadership and legacy have endured long after the war, and he rema
> Response: Winston Churchill was known for his many memorable quotes, but one

**"We shall fight on the beaches, we shall fight on the landing grounds, we sl

This quote is from his speech to the House of Commons on June 4, 1940, during

However, if I had to pick a single, even more concise quote, it would be:

**"Blood, toil, tears, and sweat."**

This was the opening phrase of his first speech as Prime Minister to the House

"I say to the House as I said to those who have joined this Government, I have

This quote has become synonymous with Churchill's leadership and resolve durin
```

Here is an example for you to try a conversation yourself. Remember to type `quit` or `exit` after you are done chatting.

```
# NBVAL_SKIP
from termcolor import cprint

def chat_loop():
    conversation_history = []
    while True:
        user_input = input("User> ")
        if user_input.lower() in ["exit", "quit", "bye"]:
            cprint("Ending conversation. Goodbye!", "yellow")
            break

        user_message = {"role": "user", "content": user_input}
        conversation_history.append(user_message)

        response = client.inference.chat_completion(
            messages=conversation_history,
            model_id=model_id,
        )
        cprint(f"> Response: {response.completion_message.content}", "cyan")

        assistant_message = {
            "role": "assistant",  # was user
            "content": response.completion_message.content,
            "stop_reason": response.completion_message.stop_reason,
        }
        conversation_history.append(assistant_message)


chat_loop()
```

```
> Response: Hello! How are you today? Is there something I can help you with (
Ending conversation. Goodbye!
```